Name: Madhav Raghu Anantharam

PUID: 040888484

First I created two helper functions to calculate the gen and kill set so I could reuse them as required.

std::set<uint32_t> AvailableExpressions::calcKillSet(Instruction* inst) -> create an empty kill set and check if inst is store or call instruction. Then loop through every tracked expression in id2Expr and its operands. Kill the expression and stop checking its operands if any of conditions are true:

- the instruction produces a value that is one of the expression's operands
- instruction is store, operand is load, mayOverlap() indicates the store instruction's address may write to the location load reads from
- instruction is call, operand is load, and escapesToCallee() says the call can reach the loaded location

then return the kill set

std::set<uint32_t> AvailableExpressions::calcGenSet(Instruction* inst) -> create an empty gen set. If the instruction is in expr2Id, insert its ID. Then return gen set.

bool AvailableExpressions::runOnFunction(Function &F) -> followed the steps that were given in the assignment pdf.

bool AvailableExpressions::isAvailableAfter(Instruction &expr, Instruction &point) -> if the expression is not tracked or the point's basic block has no IN set, return false. Else, get the expression ID and start a current set from the block's IN set. Go through the block's instructions in order and update current = gen ∪ (current − kill) with each instruction's gen and kill sets. Stop once the point instruction has been processed. Return Boolean based on if the expression's ID is still in current.

bool CommonSubexpressionElimination::runOnFunction(llvm::Function &F) -> loop until a pass makes no changes. On each pass:

Create a toBeDeleted set for redundant instructions and a loaded set for the load instructions.

Go through every basic block. Get its replacement expressions with ae.getInSet(&BB), which returns the expressions computed in the block plus the ones in its IN set.

For every instruction in the block, skip unless it is a BinaryOperator, CmpInst or GetElementPtrInst.

Set point to the instruction just before it. If it's the first instruction in the block, use the instruction itself.

Go through the replacement expressions and skip if any of these conditions are true:

- it is the same instruction
- it is already marked for deletion
- Search() finds it is not the same expression (same opcode, matching operands, or swapped operands for commutative operations and compares)
- ae.isAvailableAfter() shows it is no longer available at point

On the first instruction that passes, save the load operands into loaded. Then call replaceAllUsesWith(prevInst) so every use points to the previous expression, add the instruction to toBeDeleted, and break.

After all blocks are processed, clear toBeDeleted. Then remove any load in loaded that now has no uses. If anything was deleted, set changed and globalChanged to true and rerun ae.runOnFunction(F) so the available expressions are up to date for the next pass. Removing one expression can make another one redundant, so the loop keeps going until nothing changes. Return the final globalChanged value.

Note: when I tried to run the tests on my local machine (M5 macbook air), 12 out of the last 23 tests failed. However when I ran the code on the data.cs server, all passed without any issues. I suspect that this is due to a difference in register allocation of a macOS vs linux machine.